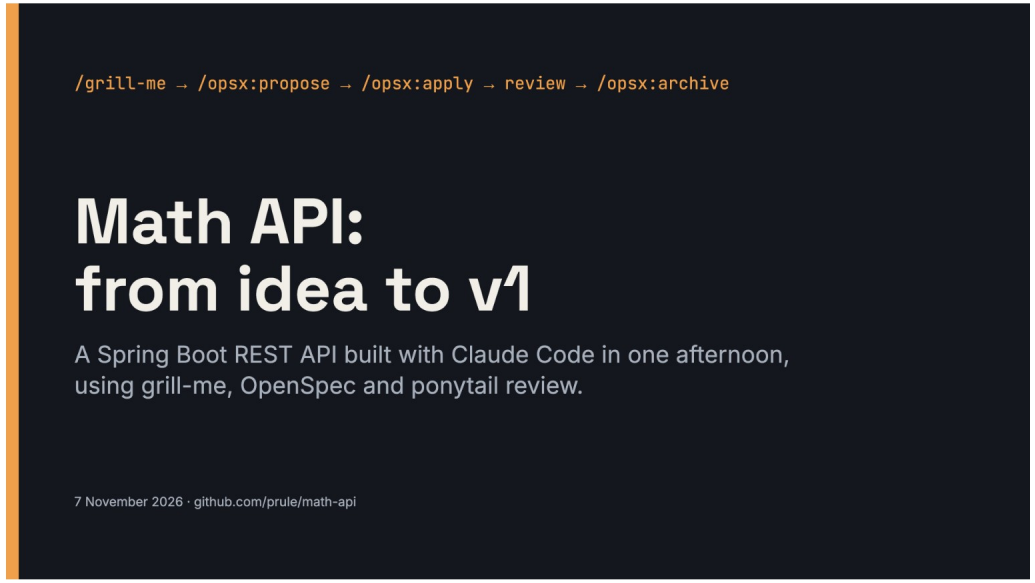
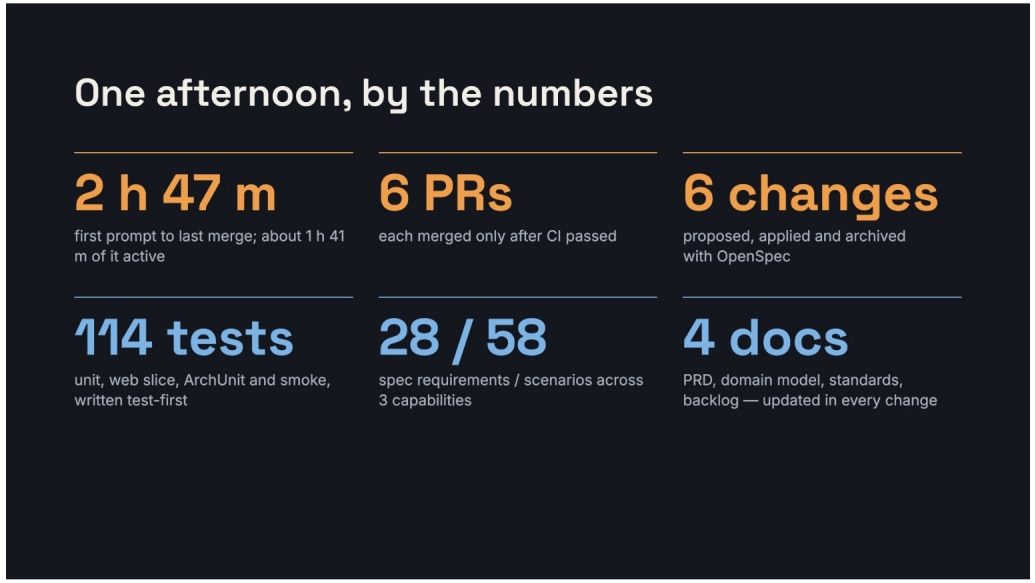
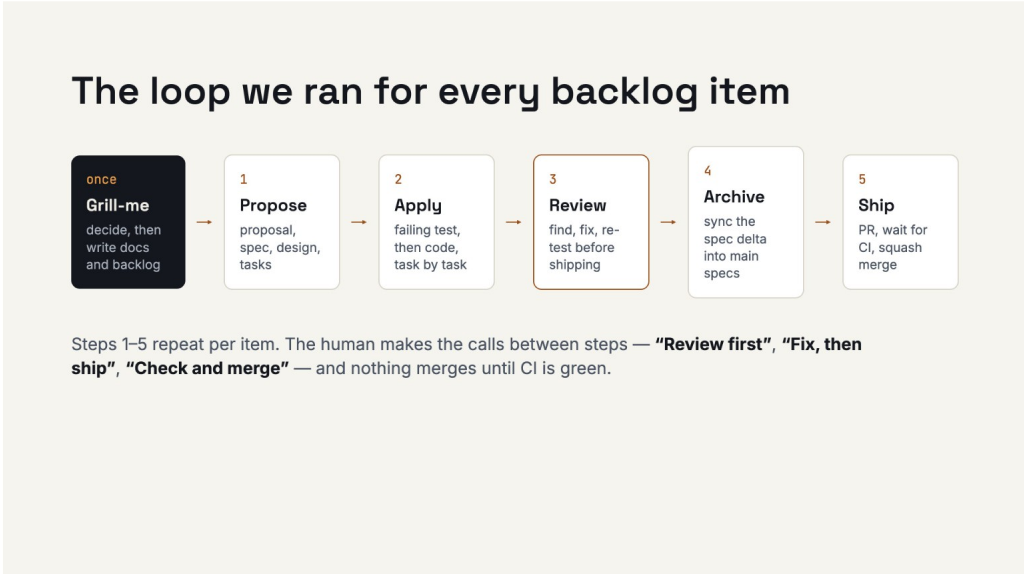


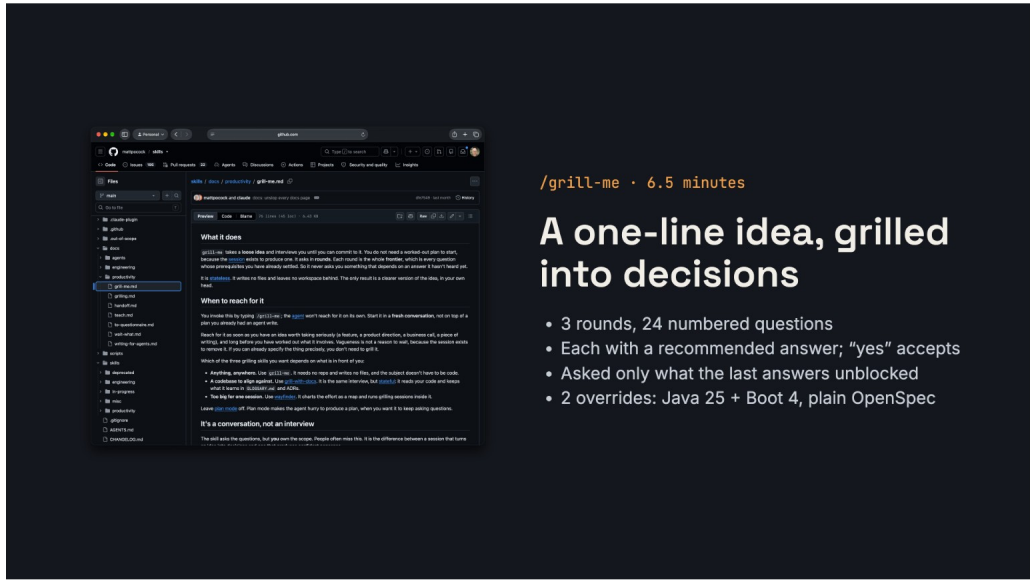
One afternoon, one session. We went from a one-line idea to a merged v1 plus a follow-up: six pull requests, each planned with OpenSpec, built test-first, reviewed, and merged only after CI passed.



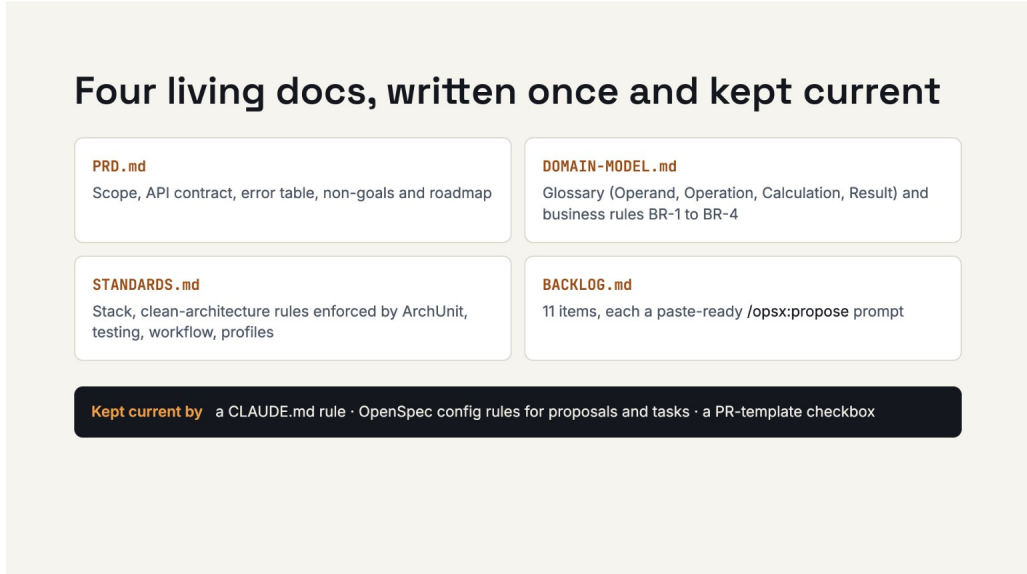
Times come from the session's own event log: first prompt at 12:37, last merge at 15:24. There was a 66-minute break between v1 and the follow-up. The 114 tests run in about 4 seconds. Production code is 404 lines across 18 Java files; tests are 732 lines.



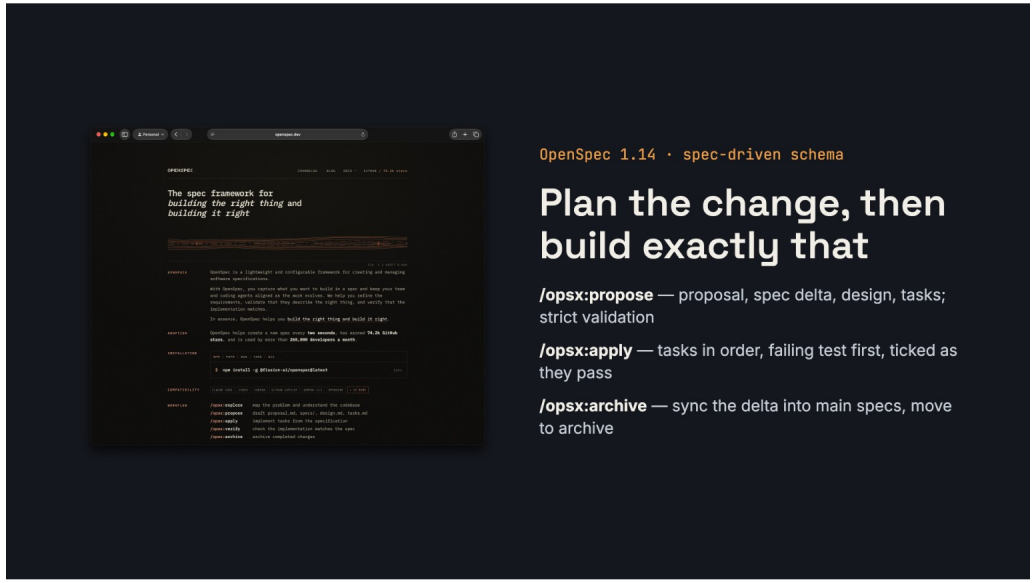
Grill-me runs once, up front. After that every backlog item went through the same five steps. The review step was optional at first; from item 3 on it became the habit. Every decision point was a one-click answer in the app, so the human stayed in control without typing much.



The starting prompt was one sentence: a Spring Boot REST API, clean architecture, the four basic operations, more to come, plus a PRD, standards and domain model kept up to date. Grill-me worked the decisions as a tree: round 1 covered operation shape, API style, number type, state, auth, stack, doc layout and how docs stay current. Later rounds covered division rounding, error format, input limits, layering, testing, naming, CI and versioning. Most answers were a single “yes”.



Grill-me asks the questions; it does not write files. Once we agreed, the four docs were written in one pass, plus CLAUDE.md, a PR template and the OpenSpec config. The config's context tells every OpenSpec artifact to read these docs first. Its rules make every proposal list which docs it updates, and every task list include those updates with a change log row. Every one of the six changes updated at least two docs.



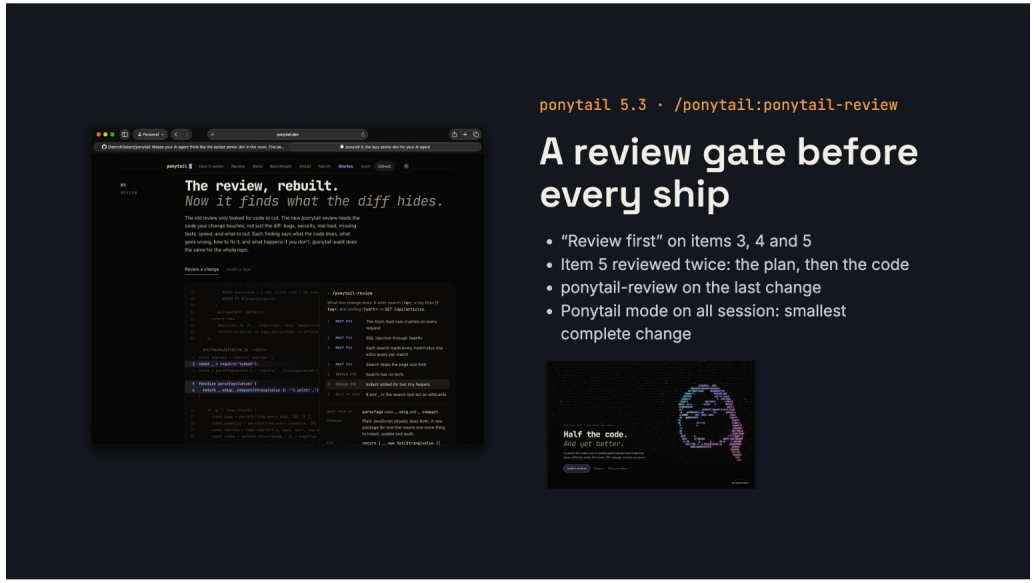
We used plain OpenSpec rather than a heavier agent pipeline. Each change folder held four artifacts. The spec delta held observable behaviour with WHEN/THEN scenarios, which became the tests. The design recorded decisions with the alternatives rejected. Tasks were written test-first, each with how to verify it. Apply paused when it found something outside the plan, such as the Swagger webjar under prod, and asked before widening scope.

Archive with sync grew the specs, change by change

Change	Capability	Delta	Reqs after
add-walking-skeleton	service-health	new, +4	4
add-exact-arithmetic	calculation	new, +10	10
add-divide	calculation	+2	12
add-operand-limits	calculation	+5	17
add-api-docs	api-documentation	new, +6	6
add-api-docs-toggle	api-documentation	2 modified, +1	7

Every sync was checked with openspec validate --strict before the change folder moved to archive/. The first sync caught a spec sentence the running app did not match, and fixed the wording.

The main specs in openspec/specs are the current truth: 28 requirements and 58 scenarios across three capabilities. Each archive copied ADDED requirements in, and replaced MODIFIED ones whole. The archived change folders keep the history of why: decisions, findings during apply and review notes.



Ponytail ran in full mode the whole session. It is why the docs toggle needed no Java code and the operation list in the API docs comes straight from the enum. Each review read the connected code, not just the diff. Each finding came with a concrete failure case and the smallest fix, and the human chose which to apply. The last review, ponytail-review, found nothing and said “Looks good. Ship.”

What the reviews caught

Review	Found	Fixed with
Divide · code	“Unknown operation” test used names we plan to add; 0 ÷ 0 untested through the API	Never-valid name; one more 422 case
Limits · code	Scale check safe only via a (long) cast: 1E+2147483648 has scale Integer.MIN_VALUE	Comment, plus a test that fails without the cast
API docs · plan	200 response would be documented as */*; generic responses could leak in	Explicit 200; exact-codes test
API docs · code	Error schema showed a “properties” field responses never contain	Removed in the customiser; schema pinned
Toggle · apply	Spring still served the Swagger UI webjar under prod	add-mappings: false; 404 test
Toggle · ponytail	Nothing: “Looks good. Ship.”	—

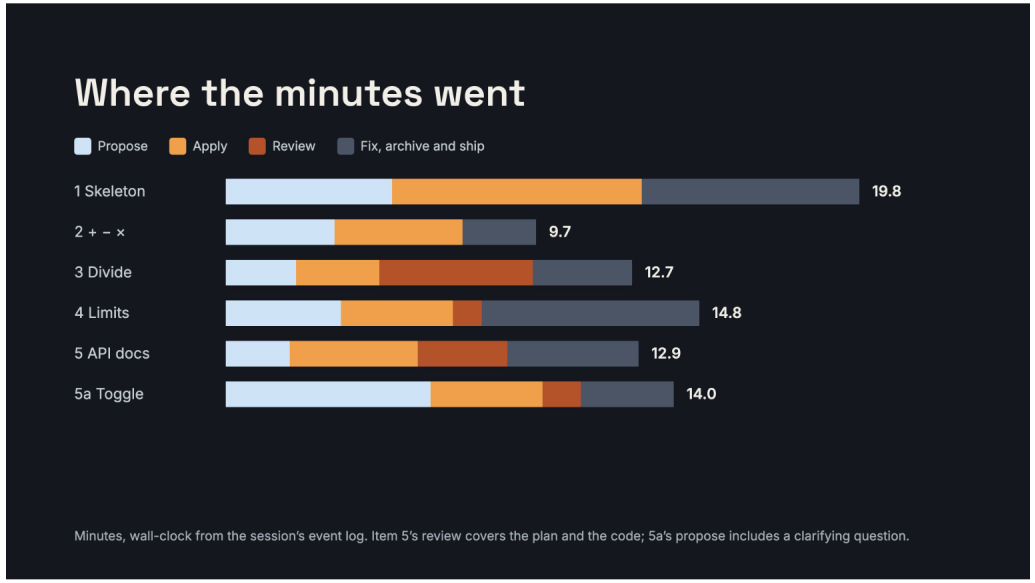
Four of the five reviews found something real, and apply caught one more. Every fix landed with a test that fails without it. None of these would have broken a build. They are the gaps between “tests pass” and “correct”: docs that misdescribe the API, a safety check one refactor away from silently disappearing, and a prod setting that left something exposed.

Backlog delivered: v1 plus one follow-up

#	Item	Review	PR	Propose → merge
1	Walking skeleton	—	#1	19.8 min
2	ADD, SUBTRACT, MULTIPLY	—	#2	9.7 min
3	DIVIDE, 422 on zero	code	#3	12.7 min
4	Operand limits (BR-1)	code	#4	14.8 min
5	OpenAPI docs	plan + code	#5	12.9 min
5a	Docs off under prod	ponytail	#6	14.0 min

Wall-clock from the “Propose” prompt to the PR merging, including human decisions and CI. Docs + backlog via grill-me: 9.3 min.

Item 1 includes creating the public GitHub repo and the first CI run. Item 2 had no review and was the fastest, at under 10 minutes. Items with a review took 13 to 15 minutes, including the fixes. Every item ran the same loop and merged with a single squashed commit.

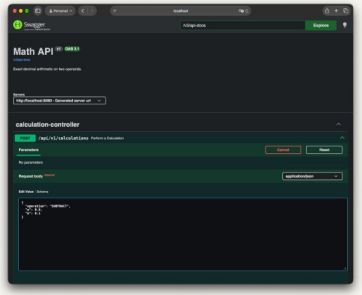


Apply, the actual coding, was 3 to 8 minutes per item. Most of each bar is the planning and verification around it. The first item's long ship bar includes creating the GitHub repo and the first CI run. For item 4, the fix was a cast comment plus tests, so the review itself was under a minute. A review plus its fix cost 2 to 8 minutes, and found something every time but the last.

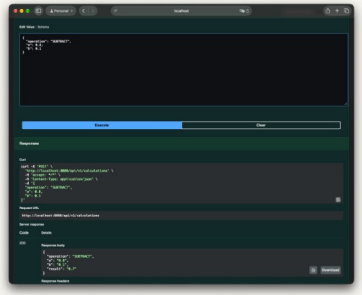


These are API list prices for claude-opus-5-5 at medium effort, counted from the session’s own transcript, one entry per model response. On a Claude subscription this is plan usage, not a bill. Each turn re-reads the whole conversation, which is why cache reads dominate: they are billed at a twentieth of the input price. That is what keeps a long, many-turn session like this affordable. The sdlc-pipeline session report showed higher totals because it counts each response once per transcript line, about 2.5 times over.

The running API, documented by itself

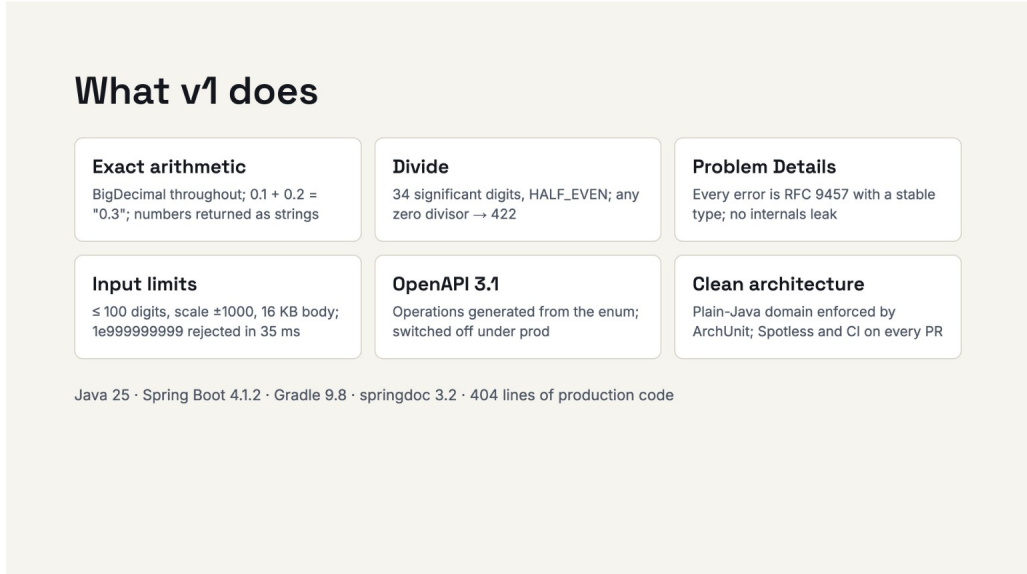


Swagger UI generated at /swagger-ui.html, OAS 3.1

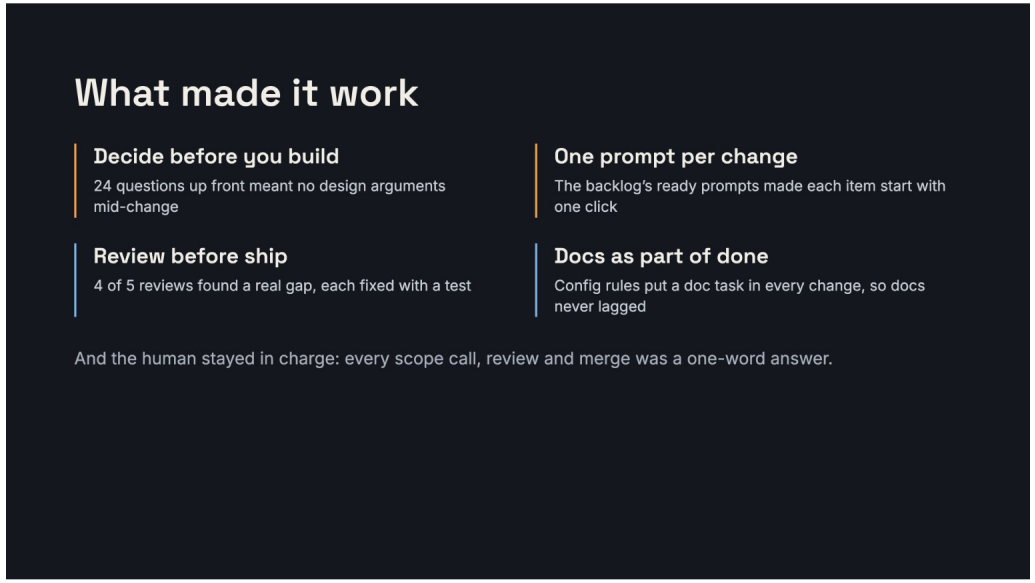


0.8 − 0.1 returns **"0.7"**, exactly

With binary floating point, 0.8 minus 0.1 gives 0.7000000000000001. The API uses BigDecimal throughout and returns numbers as strings, so clients never lose precision. The operation dropdown in Swagger UI is generated from the domain's Operation enum, so adding an operation updates the docs with no edits.



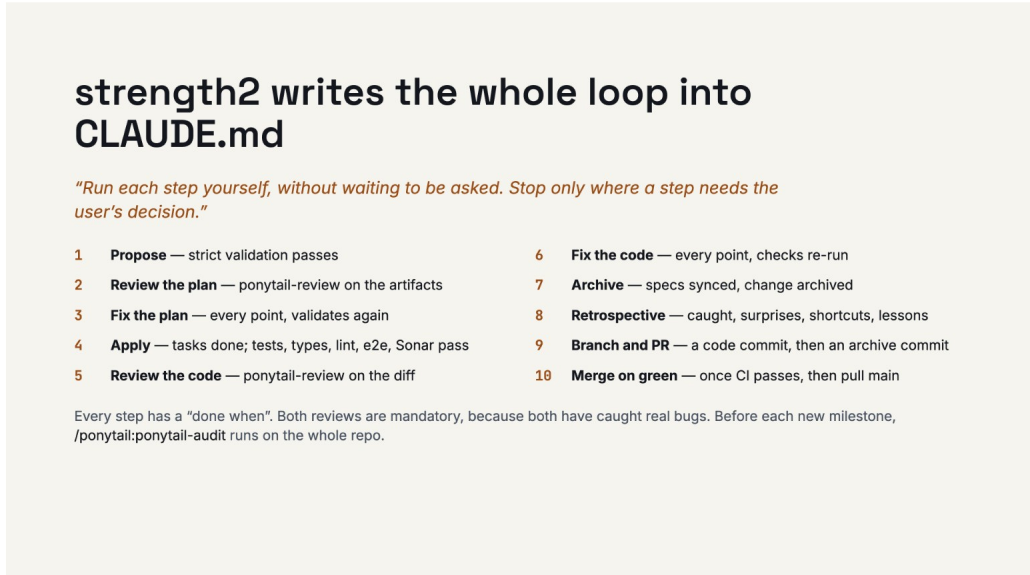
Before the limits change, a request like 1e-999999999 divided by 1e999999999 returned a 500, and extreme additions could exhaust memory. Now operands are checked as written, before any arithmetic, and Jackson’s read limits bound the parsing work. The domain package has no Spring imports, and ArchUnit fails the build if one appears.



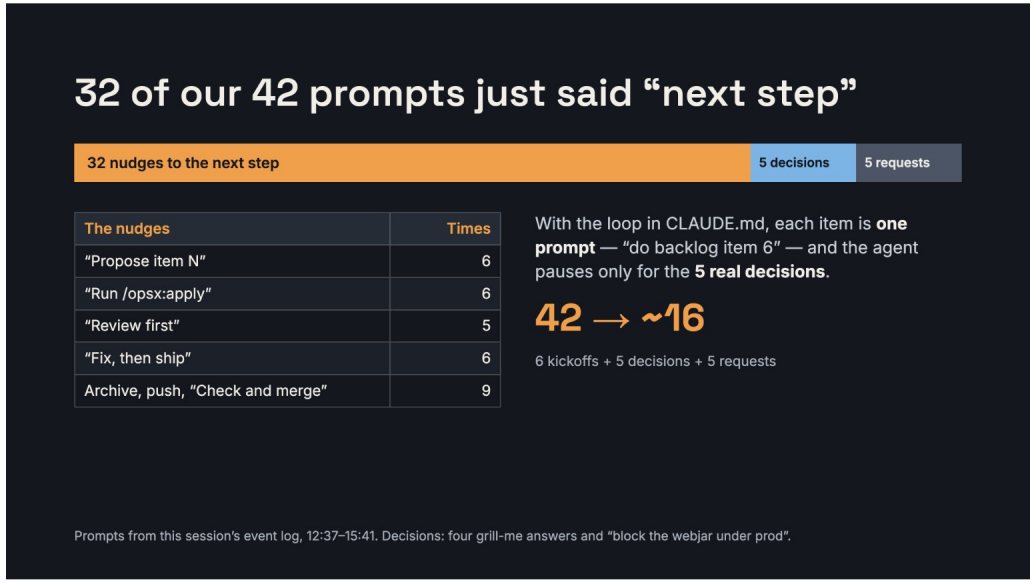
The tools each did one job. Grill-me settled what to build. OpenSpec turned each backlog item into a plan, tracked the work and kept the specs current. Ponytail kept the code small and the reviews concrete. CI gated every merge. The person made every decision that mattered: Java 25, plain OpenSpec, blocking the webjar under prod, and when to ship.



Each later item needs a PRD update first, because each moves something out of the non-goals or roadmap. Item 6 must also update the “Unsupported name” spec scenario, which uses POWER, and the unknown-operation example in the API docs.



This is the strength2 project's CLAUDE.md: 29 lines. It is the same toolset as math-api — OpenSpec, ponytail, a PR skill, CI — but the loop is written down as instructions to the agent, not driven by the person one click at a time. The agent stops only for a material ambiguity while proposing, or a choice a review raises. strength2 found that both reviews caught real bugs, such as a restore that could wipe all data and an e2e journey that could not run.



Every nudge was a good checkpoint, but none needed judgment. “Review first” was always the right call, and “Fix, then ship” always meant apply every finding. Written into CLAUDE.md, those become defaults the agent follows. The person still answers ambiguities and review choices that are genuinely theirs. One caveat: in this app the agent does not poll CI, so a CI wait may still need a “check and merge” nudge unless the PR monitor is switched on. The estimate of about 16 assumes that.

What the strength2 loop would add to ours

Practice	math-api today	With the CLAUDE.md loop
Plan review	1 of 6 changes	Every change, before apply
Code review	4 of 6, when asked	Every change, fix every point
Retrospective	Rebuilt by hand for this deck	retrospective.md in each change
Shortcuts	One, found by grep	ponytail-debt ledger per change
Commits	One squashed commit	Code commit + archive commit
Milestones	No audit	ponytail-audit before each one

The deck's "What the reviews caught" and "Where the minutes went" slides would come straight from the retrospectives.

The biggest gains are consistency and memory. Items 1 and 2 shipped without a review, and only item 5 had its plan reviewed — yet that plan review caught the */* response type before any code was written. Retrospectives turn what we reconstructed from logs today into a file written at the moment of each change. The ponytail-debt ledger keeps shortcuts like the old ArchUnit empty-layer allowance visible until they are paid off.



A draft, not yet in the repo. It folds strength2’s ten steps into eight by pairing each review with its fix. It uses this repo’s single check command, and squash merges as we did today; strength2 rebases two commits. Item 6 is the natural point to start: it opens the post-v1 backlog, so the milestone audit runs first.